

SQL – Data Manipulation Language

Carla Teixeira Lopes

Bases de Dados

Licenciatura em Engenharia Informática e Computação, FEUP+FCUP

Based on Jennifer Widom slides

Agenda

Introduction

The JOIN family of operators

Basic SELECT Statement

Aggregation

Table Variables and Set Operators

Null values

Subqueries in WHERE clauses

Data Modification statements

Subqueries in FROM and SELECT clauses

SQL

Stands for **S**tructured **Q**uery **L**anguage

Pronounced “sequel”

Supported by all major commercial database systems

Standardized – many features over time

Interactive via GUI or prompt, or embedded in programs

Declarative, based on relational algebra

SQL History

1970 “A Relational Model of Data for Large Shared Data Banks” by Edgar Codd

Early **70's** SEQUEL Developed at IBM by Donald Chamberlin e Raymond Boyce

1979 First commercial version by Relational Software (now Oracle)

1986 SQL-86 and SQL-87. Ratified by ANSI and ISO

1989 SQL-89 1992 SQL-92. Also known as SQL2

1999 SQL:1999. Also known as SQL3. Includes regular expressions, recursive queries, triggers, non-scalar data types and some object-oriented expressions

2003 SQL:2003 XML support and auto-generated values

2006 SQL:2006 XQuery support

2008 SQL:2008; **2011** SQL:2011; **2016** SQL: 2016; **2019** SQL: 2019–2020; **2023** SQL: 2023

SQL is a ...

Data Definition Language (DDL)

- Define relational schemata

- Create/alter/delete tables and their attributes

Data Manipulation Language (DML)

- Insert/delete/modify tuples in tables

- Query one or more tables

Standard

Many standards out there

Database management systems implement something similar, but not identical to the standard for SQL

These slides will try to adhere to the standard as much as possible

Primarily the SQL2 standard and some constructs from the SQL3 standard

Sometimes we'll talk specifically about SQL as understood by SQLite

Agenda

Introduction

The JOIN family of operators

Basic SELECT Statement

Aggregation

Table Variables and Set Operators

Null values

Subqueries in WHERE clauses

Data Modification statements

Subqueries in FROM and SELECT clauses

The Basic SELECT Statement (SFW)

SELECT $A_1, A_2, \dots, A_n$	→	What to return
FROM $R_1, R_2, \dots, R_m$	→	Identifies the relations to query
WHERE condition	→	Combines and filters relations

What's the equivalent in Relational Algebra?

$$\pi_{A_1, \dots, A_n} (\sigma_{condition} (R_1 \times \dots \times R_m))$$

The result is a relation with the schema $A_1, A_2, \dots, A_n$

SQL is compositional

Relational query languages are compositional

When a query is run over relations, the result is a relation

The schema of the obtained relation is the set of attributes that are returned

The output of one query can be used as the input to another (nesting)

This is extremely powerful

College Admission Database

Apply

<u>sID</u>	<u>cName</u>	<u>major</u>	<u>dec</u>
123	Stanford	CS	Y
123	Stanford	EE	N
123	Berkeley	CS	Y
123	Cornell	EE	Y
234	Berkeley	biology	N
345	MIT	bioengineering	Y
345	Cornell	bioengineering	N
345	Cornell	CS	Y
345	Cornell	EE	N
678	Stanford	history	Y
987	Stanford	CS	Y
987	Berkeley	CS	Y
876	Stanford	CS	Y
876	MIT	biology	Y
876	MIT	marine biology	N
765	Stanford	history	Y
765	Cornell	history	N
765	Cornell	psychology	Y
543	MIT	CS	N

College

<u>cName</u>	<u>state</u>	<u>enr</u>
Stanford	CA	15000
Berkeley	CA	36000
MIT	MA	10000
Cornell	NY	21000

Student

<u>sID</u>	<u>sName</u>	<u>GPA</u>	<u>HS</u>
123	Amy	3.9	1000
234	Bob	3.6	1500
345	Craig	3.5	500
456	Doris	3.9	1000
567	Edward	2.9	2000
678	Fay	3.8	200
789	Gary	3.4	800
987	Helen	3.7	800
876	Irene	3.9	400
765	Jay	2.9	1500
654	Amy	3.9	1000
543	Craig	3.4	2000

SQL query with one relation

```
SELECT sID, sName, GPA
FROM Student
WHERE GPA > 3.6;
```

sID	sName	GPA
123	Amy	3.9
456	Doris	3.9
678	Fay	3.8
987	Helen	3.7
876	Irene	3.9
654	Amy	3.9

Not necessary to include GPA in the result even if we filter on the GPA

College(cName, state, enr)

Student(sID, sName, GPA, sizeHS)

Apply(sID, cName, major, decision)

A Few Details

SQL commands are case insensitive

Same: SELECT, Select, select

Same: Student, student

Values are case sensitive

Different: 'Stanford', 'stanford'

Use single quotes for constants

'abc' - yes

"abc" - no

SQL query combining two relations

```
SELECT sName, major
FROM Student, Apply
WHERE Student.sID=Apply.sID;
```

This would happen automatically in a natural join of RA

What does it compute?

College(cName, state, enr)
Student(sID, sName, GPA, sizeHS)
Apply(sID, cName, major, decision)

Duplicate values

sName	major
Amy	CS
Amy	EE
Amy	CS
Amy	EE
Bob	biology
Craig	bioengineering
Craig	bioengineering
Craig	CS
Craig	EE
Fay	history
Helen	CS
Helen	CS
Irene	CS
Irene	biology
Irene	marine biology
Jay	history
Jay	history
Jay	psychology

SQL query excluding duplicate values

```
SELECT DISTINCT sName, major
FROM Student, Apply
WHERE Student.sID=Apply.sID;
```

sName	major
Amy	CS
Amy	EE
Bob	biology
Craig	bioengineering
Craig	CS
Craig	EE
Fay	history
Helen	CS
Irene	CS
Irene	biology
Irene	marine biology
Jay	history
Jay	psychology

No duplicate values



College(<u>cName</u> , state, enr)
Student(<u>sID</u> , sName, GPA, sizeHS)
Apply(<u>sID</u> , <u>cName</u> , <u>major</u> , decision)

Another SQL query combining two relations

```
SELECT sName, GPA, decision
FROM   Student, Apply
WHERE  Student.sID=Apply.sID AND sizeHS<1000 AND
       major='CS' AND cName='Stanford';
```

What does it compute?

sName	GPA	decision
Helen	3.7	Y
Irene	3.9	N

College(cName, state, enr)

Student(sID, sName, GPA, sizeHS)

Apply(sID, cName, major, decision)

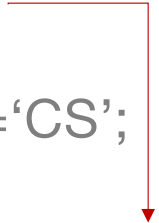
One more SQL query combining two relations

```
SELECT cName
FROM   College, Apply
WHERE  College.cName=Apply.cName AND enr>20000 AND major='CS';
```

What does it compute?

How can we correct it?

SQLite Error: ambiguous
column name: cName



College(<u>cName</u> , state, enr)
Student(<u>sID</u> , sName, GPA, sizeHS)
Apply(<u>sID</u> , <u>cName</u> , <u>major</u> , decision)

One more SQL query combining two relations

```
SELECT College.cName
FROM   College, Apply
WHERE  College.cName=Apply.cName AND enr>20000 AND major='CS';
```

cName
Berkeley
Berkeley
Cornell



How can we eliminate duplicates?

College(<u>cName</u> , state, enr)
Student(<u>sID</u> , sName, GPA, sizeHS)
Apply(<u>sID</u> , <u>cName</u> , <u>major</u> , decision)

Order of the results

SQL is based on an unordered model

The order of the results may change each time we run a query

We can ask for a result to be sorted, in ascending or descending order, by an attribute or set of attributes

`ORDER BY <attributes> ASC`

`ORDER BY <attributes> DESC`

Ordering is ascending, unless you specify the DESC keyword

Order of the results

College(cName, state, enr)
Student(sID, sName, GPA, sizeHS)
Apply(sID, cName, major, decision)

```
SELECT Student.sID, sName, GPA,  
       Apply.cName, enr  
FROM   Student, College, Apply  
WHERE  Student.sID=Apply.sID AND  
       Apply.cName=College.cName;
```

sID	sName	GPA	cName	enr
123	Amy	3.9	Stanford	15000
123	Amy	3.9	Stanford	15000
123	Amy	3.9	Berkeley	36000
123	Amy	3.9	Cornell	21000
234	Bob	3.6	Berkeley	36000
345	Craig	3.5	MIT	10000
345	Craig	3.5	Cornell	21000
345	Craig	3.5	Cornell	21000
345	Craig	3.5	Cornell	21000
678	Fay	3.8	Stanford	15000
987	Helen	3.7	Stanford	15000

```
SELECT Student.sID, sName, GPA,  
       Apply.cName, enr  
FROM   Student, College, Apply  
WHERE  Student.sID=Apply.sID AND  
       Apply.cName=College.cName  
  
ORDER BY GPA DESC;
```

sID	sName	GPA	cName	enr
123	Amy	3.9	Stanford	15000
123	Amy	3.9	Stanford	15000
123	Amy	3.9	Berkeley	36000
123	Amy	3.9	Cornell	21000
876	Irene	3.9	Stanford	15000
876	Irene	3.9	MIT	10000
876	Irene	3.9	MIT	10000
678	Fay	3.8	Stanford	15000
987	Helen	3.7	Stanford	15000
987	Helen	3.7	Berkeley	36000

Order of the results

```
SELECT Student.sID, sName, GPA, Apply.cName, enr
FROM   Student, College, Apply
WHERE  Student.sID=Apply.sID AND Apply.cName=College.cName
ORDER BY GPA DESC, enr;
```

sID	sName	GPA	cName	enr
876	Irene	3.9	MIT	10000
876	Irene	3.9	MIT	10000
123	Amy	3.9	Stanford	15000
123	Amy	3.9	Stanford	15000
876	Irene	3.9	Stanford	15000
123	Amy	3.9	Cornell	21000
123	Amy	3.9	Berkeley	36000
678	Fay	3.8	Stanford	15000
987	Helen	3.7	Stanford	15000

Descending GPA as primary sort order and,
within each of those, ascending enrollment

College(cName, state, enr)
Student(sID, sName, GPA, sizeHS)
Apply(sID, cName, major, decision)

Like operator: pattern matching on strings

Built-in operator that allows string matching on attribute values

```
SELECT sID, major  
FROM Apply  
WHERE major like '%bio%';
```

sID	major
234	biology
345	bioengineering
345	bioengineering
876	biology
876	marine biology

Match any major containing bio

% = any sequence of characters

_ = any single character

College(cName, state, enr)
Student(sID, sName, GPA, sizeHS)
Apply(sID, cName, major, decision)

Selecting all attributes

```
SELECT *  
FROM Apply  
WHERE major like '%bio%';
```

sID	cName	major	decision
234	Berkeley	biology	N
345	MIT	bioengineering	Y
345	Cornell	bioengineering	N
876	MIT	biology	Y
876	MIT	marine biology	N

College(cName, state, enr)

Student(sID, sName, GPA, sizeHS)

Apply(sID, cName, major, decision)

Arithmetic within SQL clauses

```
SELECT sID, sName, GPA, sizeHS, GPA*(sizeHS/1000)
FROM Student;
```

sID	sName	GPA	HS	GPA*(HS/1000)
123	Amy	3.9	1000	3.9
234	Bob	3.6	1500	5.4
345	Craig	3.5	500	1.75
456	Doris	3.9	1000	3.9
567	Edward	2.9	2000	5.8
678	Fay	3.8	200	0.76
789	Gary	3.4	800	2.72
987	Helen	3.7	800	2.96
876	Irene	3.9	400	1.56
765	Jay	2.9	1500	4.35
654	Amy	3.9	1000	3.9
543	Craig	3.4	2000	6.8

Boosts GPA if student is from a big high school and reduces it, if he is from a small one

Can we improve this result?

College(cName, state, enr)
Student(sID, sName, GPA, sizeHS)
Apply(sID, cName, major, decision)

Renaming columns

```
SELECT sID, sName, GPA, HS, GPA*(HS/1000) AS scaledGPA  
FROM Student;
```

sID	sName	GPA	HS	scaledGPA
123	Amy	3.9	1000	3.9
234	Bob	3.6	1500	5.4
345	Craig	3.5	500	1.75
456	Doris	3.9	1000	3.9
567	Edward	2.9	2000	5.8
678	Fay	3.8	200	0.76
789	Gary	3.4	800	2.72
987	Helen	3.7	800	2.96
876	Irene	3.9	400	1.56
765	Jay	2.9	1500	4.35
654	Amy	3.9	1000	3.9
543	Craig	3.4	2000	6.8

College(cName, state, enr)

Student(sID, sName, GPA, sizeHS)

Apply(sID, cName, major, decision)

Agenda

Introduction

The JOIN family of operators

Basic ~~SELECT~~ Statement

Aggregation

Table Variables and Set Operators

Null values

Subqueries in WHERE clauses

Data Modification statements

Subqueries in FROM and SELECT clauses

Table variables

Used in the FROM clause for two purposes

- Make queries more readable

- Rename relations when we have two instances of the same relation

```
SELECT Student.sID, sName, GPA, Apply.cName, enr
FROM   Student, College, Apply
WHERE  Student.sID=Apply.sID AND Apply.cName=College.cName;
```

```
SELECT S.sID, sName, GPA, A.cName, enr
FROM   Student S, College C, Apply A
WHERE  S.sID=A.sID AND A.cName=C.cName;
```



Not changing the result, only
making the query more readable

Table variables

How to list the pairs of students who have the same GPA?

Student(sID, sName, GPA, sizeHS)

```
SELECT S1.sID, S1.sName, S1.GPA, S2.sID, S2.sName, S2.GPA
FROM   Student S1, Student S2
WHERE  S1.GPA=S2.GPA;
```

How to list only pairs
of different students?

S1.sID	S1.sName	S1.GPA	S2.sID	S2.sName	S2.GPA
123	Amy	3.9	123	Amy	3.9
123	Amy	3.9	456	Doris	3.9
123	Amy	3.9	876	Irene	3.9
123	Amy	3.9	654	Amy	3.9
234	Bob	3.6	234	Bob	3.6
345	Craig	3.5	345	Craig	3.5
456	Doris	3.9	123	Amy	3.9
456	Doris	3.9	456	Doris	3.9
...	...	...	...	...	...

Table variables

```
SELECT S1.sID, S1.sName, S1.GPA, S2.sID, S2.sName, S2.GPA
FROM   Student S1, Student S2
WHERE  S1.GPA=S2.GPA AND S1.sID <> S2.sID;
```

How to exclude the same pairs
in different order?

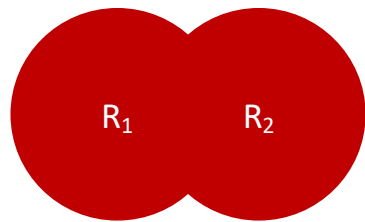
S1.sID	S1.sName	S1.GPA	S2.sID	S2.sName	S2.GPA
123	Amy	3.9	456	Doris	3.9
123	Amy	3.9	876	Irene	3.9
123	Amy	3.9	654	Amy	3.9
456	Doris	3.9	123	Amy	3.9
456	Doris	3.9	876	Irene	3.9
456	Doris	3.9	654	Amy	3.9
567	Edward	2.9	765	Jay	2.9
...	...	...	...	...	...

Table variables

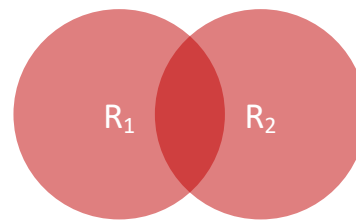
```
SELECT S1.sID, S1.sName, S1.GPA, S2.sID, S2.sName, S2.GPA
FROM   Student S1, Student S2
WHERE  S1.GPA=S2.GPA AND S1.sID < S2.sID;
```

S1.sID	S1.sName	S1.GPA	S2.sID	S2.sName	S2.GPA
123	Amy	3.9	456	Doris	3.9
123	Amy	3.9	876	Irene	3.9
123	Amy	3.9	654	Amy	3.9
456	Doris	3.9	876	Irene	3.9
456	Doris	3.9	654	Amy	3.9
567	Edward	2.9	765	Jay	2.9
654	Amy	3.9	876	Irene	3.9
543	Craig	3.4	789	Gary	3.4

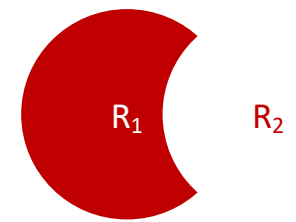
Set operators



Union



Intersect



Except

Sets, Bags and Lists

Sets

Only one occurrence of each element

Unordered elements

Bags (or multisets)

More than one occurrence of an element

Unordered elements and their occurrences

Lists

More than one occurrence of an element

Occurrences are ordered

Recall Bags

Multiset X

Tuple
(1, a)
(1, a)
(1, b)
(2, c)
(2, c)
(2, c)
(1, d)
(1, d)



Multiset X

Tuple	$\lambda(X)$
(1, a)	2
(1, b)	1
(2, c)	3
(1, d)	2

$\lambda(X)$ = “Count of tuple in X”

In a set all counts are (0,1).

Union as a bag operation

Multiset X

Tuple	$\lambda(X)$
(1, a)	2
(1, b)	0
(2, c)	3
(1, d)	0

U

Multiset Y

Tuple	$\lambda(Y)$
(1, a)	5
(1, b)	1
(2, c)	2
(1, d)	2

=

Multiset Z

Tuple	$\lambda(Z)$
(1, a)	7
(1, b)	1
(2, c)	5
(1, d)	2

$$\lambda(Z) = \lambda(X) + \lambda(Y)$$

Intersect as a bag operation

Multiset X

Tuple	$\lambda(X)$
(1, a)	2
(1, b)	0
(2, c)	3
(1, d)	0

$\cap$

Multiset Y

Tuple	$\lambda(Y)$
(1, a)	5
(1, b)	1
(2, c)	2
(1, d)	2

$=$

Multiset Z

Tuple	$\lambda(Z)$
(1, a)	2
(1, b)	0
(2, c)	2
(1, d)	0

$$\lambda(Z) = \min(\lambda(X), \lambda(Y))$$

Except as a bag operation

Multiset X

Tuple	$\lambda(X)$
(1, a)	2
(1, b)	0
(2, c)	3
(1, d)	0

—

Multiset Y

Tuple	$\lambda(Y)$
(1, a)	5
(1, b)	1
(2, c)	2
(1, d)	2

=

Multiset Z

Tuple	$\lambda(Z)$
(1, a)	0
(1, b)	0
(2, c)	1
(1, d)	0

If $\lambda(X) > \lambda(Y)$

$$\lambda(Z) = \lambda(X) - \lambda(Y)$$

Else

0

Union operator

College(cName, state, enr)

Student(sID, sName, GPA, sizeHS)

Apply(sID, cName, major, decision)

```
SELECT cName FROM College
UNION
SELECT sName FROM Student;
```

In SQL, the two sides of the union don't have to be the same

How to unify the two schemas?

```
SELECT cName AS name FROM College
UNION
SELECT sName AS name FROM Student;
```

cName
Amy
Berkeley
Bob
Cornell
Craig
Doris
Edward
Fay
Gary
Helen
Irene
Jay
MIT
Stanford

Union operator

College(cName, state, enr)

Student(sID, sName, GPA, sizeHS)

Apply(sID, cName, major, decision)

By default, in SQL, the union operator eliminates duplicates

If we want to have duplicates in our result

```
SELECT cName AS name FROM College
```

UNION ALL

```
SELECT sName AS name FROM Student;
```

Result is not sorted anymore

Why?

How can we sort the result?

```
SELECT cName AS name FROM College
UNION ALL
SELECT sName AS name FROM Student
ORDER BY name;
```

name
Stanford
Berkeley
MIT
Cornell
Amy
Bob
Craig
Doris
Edward
Fay
Gary
Helen
Irene
Jay
Amy
Craig

Intersect operator

College(cName, state, enr)

Student(sID, sName, GPA, sizeHS)

Apply(sID, cName, major, decision)

```
SELECT sID FROM Apply WHERE major='CS'  
INTERSECT  
SELECT sID FROM Apply WHERE major='EE';
```

sID
123
345

Some DBMS don't support the intersect operator

How to rewrite this query without the intersect operator?

Intersect operator

College(cName, state, enr)

Student(sID, sName, GPA, sizeHS)

Apply(sID, cName, major, decision)

```
SELECT A1.sID
FROM Apply A1, Apply A2
WHERE A1.sID = A2.sID AND A1.major = 'CS' AND A2.major = 'EE';
```

Duplicates are not eliminated now

How to eliminate them?

```
SELECT DISTINCT A1.sID
FROM Apply A1, Apply A2
WHERE A1.sID = A2.sID AND A1.major = 'CS' AND A2.major = 'EE';
```

A1.sID
123
123
123
123
345

Except operator

College(cName, state, enr)

Student(sID, sName, GPA, sizeHS)

Apply(sID, cName, major, decision)

It's called *difference* in Relational Algebra

```
SELECT sID FROM Apply WHERE major='CS'  
EXCEPT  
SELECT sID FROM Apply WHERE major='EE';
```

sID
543
876
987

Some DBMS don't support the except operator

How to rewrite this query without the except operator?

Except operator

College(cName, state, enr)

Student(sID, sName, GPA, sizeHS)

Apply(sID, cName, major, decision)

```
SELECT DISTINCT A1.sID
FROM Apply A1, Apply A2
WHERE A1.sID=A2.sID AND A1.major='CS' AND A2.major<>'EE';
```

A1.sID
123
345
543
876
987

Finding one pair that satisfies this doesn't mean there's not another pair with the same student where he applied to 'CS' and 'EE'

This query is finding students who applied to 'CS'

Not possible to rewrite the except query with the operators seen until now

Agenda

Introduction

The JOIN family of operators

Basic SELECT Statement

Aggregation

Table Variables and Set Operators

Null values

Subqueries in WHERE clauses

Data Modification statements

Subqueries in FROM and SELECT clauses

Subqueries in WHERE

SELECT $A_1, A_2, \dots, A_n$

FROM $R_1, R_2, \dots, R_m$

WHERE condition  Expressions involving subqueries

Subqueries are nested SELECT statements

Subqueries generate sets that are used for comparison

A first example

College(cName, state, enr)

Student(sID, sName, GPA, sizeHS)

Apply(sID, cName, major, decision)

```
SELECT sID, sName
FROM Student
WHERE sID in
    (SELECT sID FROM Apply WHERE major='CS');
```

sID	sName
123	Amy
345	Craig
987	Helen
876	Irene
543	Craig

Subquery that finds the IDs of students who have applied to a major in CS

IDs and names of students who have applied to a major in CS at some college

How can we do this query without the subquery?

A first example

College(cName, state, enr)

Student(sID, sName, GPA, sizeHS)

Apply(sID, cName, major, decision)

```
SELECT DISTINCT Student.sID, sName
FROM   Student, Apply
WHERE  Student.sID = Apply.sID and major='CS';
```

Student.sID	sName
123	Amy
345	Craig
987	Helen
876	Irene
543	Craig

Could we write sID instead of Student.sID?

Why is DISTINCT necessary here and not in the previous query?

A second example - duplicates

College(cName, state, enr)

Student(sID, sName, GPA, sizeHS)

Apply(sID, cName, major, decision)

```
SELECT sName
FROM Student
WHERE sID in
      (SELECT sID FROM Apply WHERE major='CS');
```

sName
Amy
Craig
Helen
Irene
Craig

Names of students who have applied to a major in CS at some college

A second example - duplicates

College(cName, state, enr)

Student(sID, sName, GPA, sizeHS)

Apply(sID, cName, major, decision)

```
SELECT DISTINCT sName
FROM   Student, Apply
WHERE  Student.sID = Apply.sID and major='CS';
```

sName
Amy
Craig
Helen
Irene

Why is the result different from the one in the previous query?

The two different Craigs turned in one result

Third example - duplicates

College(cName, state, enr)

Student(sID, sName, GPA, sizeHS)

Apply(sID, cName, major, decision)

Can we list the GPA of students who have applied to a major in CS at some college using a subquery?

```
SELECT GPA
FROM Student
WHERE sID in
      (SELECT sID FROM Apply WHERE major='CS');
```

GPA
3.9
3.5
3.7
3.9
3.4

Third example - duplicates

College(cName, state, enr)

Student(sID, sName, GPA, sizeHS)

Apply(sID, cName, major, decision)

Can we list the GPA of students who have applied to a major in CS at some college **without** using a subquery?

No

```
SELECT DISTINCT GPA
FROM   Student, Apply
WHERE  Student.sID = Apply.sID and major='CS';
```

GPA
3.9
3.5
3.7
3.4



The same problem as in previous example

Fourth example

College(cName, state, enr)

Student(sID, sName, GPA, sizeHS)

Apply(sID, cName, major, decision)

Can we now write a query to list the students who have applied to a major in CS but not to a major in EE, without using the except operator?

```
SELECT sID, sName
FROM Student
WHERE sID in (select sID from Apply where major='CS') AND
        sID not in (select sID from Apply where major='EE');
```

```
SELECT sID, sName
FROM Student
WHERE sID in (select sID from Apply where major='CS') AND
        not sID in (select sID from Apply where major='EE');
```

Equivalent

sID	sName
987	Helen
876	Irene
543	Craig

Exists operator

College(cName, state, enr)

Student(sID, sName, GPA, sizeHS)

Apply(sID, cName, major, decision)

Checks whether a subquery is empty or not

Write a query that finds all colleges, such that there's some other college that is in the same state

<u>cName</u>	state	enr
Stanford	CA	15000
Berkeley	CA	36000
MIT	MA	10000
Cornell	NY	21000



<u>cName</u>	state
Stanford	CA
Berkeley	CA

Exists operator

College(cName, state, enr)

Student(sID, sName, GPA, sizeHS)

Apply(sID, cName, major, decision)

```
SELECT cName, state
FROM College C1
WHERE exists (select * from College C2
              where C2.state=C1.state);
```

Correlated reference

cName	state
Stanford	CA
Berkeley	CA
MIT	MA
Cornell	NY

What's the problem?

Every college is in the same state as itself

Exists operator

College(cName, state, enr)

Student(sID, sName, GPA, sizeHS)

Apply(sID, cName, major, decision)

```
SELECT cName, state
FROM College C1
WHERE exists (select * from College C2
              where C2.state=C1.state and C1.cName<>C2.cName);
```

cName	state
Stanford	CA
Berkeley	CA

Getting the largest value

College(cName, state, enr)

Student(sID, sName, GPA, sizeHS)

Apply(sID, cName, major, decision)

Find the college that has the largest enrollment

```
SELECT cName
FROM College C1
WHERE not exists (select * from College C2
                  where C2.enr > C1.enr);
```

Find all colleges where there does not exist another college whose enrollment is higher than the first college

cName

Berkeley

To look for something that's the largest or the smallest

Getting the largest value – example 2

College(cName, state, enr)

Student(sID, sName, GPA, sizeHS)

Apply(sID, cName, major, decision)

Find the student with the highest GPA

```
SELECT sName
FROM Student C1
WHERE not exists (select * from Student C2
                  where C2.GPA > C1.GPA);
```

sName
Amy
Doris
Irene
Amy

Getting the largest value – example 2

College(cName, state, enr)

Student(sID, sName, GPA, sizeHS)

Apply(sID, cName, major, decision)

Can we find the student with the highest GPA without subqueries?

```
SELECT S1.sName, S1.GPA  
FROM Student S1, Student S2  
WHERE S1.GPA > S2.GPA;
```



Finds all students such that there is some other student whose GPA is lower, that is, all students except those who have the lowest GPA



Does this work?



Cannot do it using joins

Alternatives to Intersect and Except

```
(SELECT R.A, R.B  
FROM R)  
  
INTERSECT  
  
(SELECT S.A, S.B  
FROM S)
```



```
SELECT R.A, R.B  
FROM R  
WHERE EXISTS(  
    SELECT *  
    FROM S  
    WHERE R.A=S.A AND R.B=S.B)
```

```
(SELECT R.A, R.B  
FROM R)  
  
EXCEPT  
  
(SELECT S.A, S.B  
FROM S)
```



```
SELECT R.A, R.B  
FROM R  
WHERE NOT EXISTS(  
    SELECT *  
    FROM S  
    WHERE R.A=S.A AND R.B=S.B)
```

All operator

College(cName, state, enr)

Student(sID, sName, GPA, sizeHS)

Apply(sID, cName, major, decision)

Checks whether the value has a certain relationship with **all** the results of the subquery

```
SELECT sName
FROM Student
WHERE GPA >= all (select GPA from Student);
```

sName
Amy
Doris
Irene
Amy

All operator

College(cName, state, enr)

Student(sID, sName, GPA, sizeHS)

Apply(sID, cName, major, decision)

```
SELECT sName
FROM Student S1
WHERE GPA > all (select GPA from Student S2
                 where S2.sID <> S1.sID);
```

sName

Does this work?

The query would be correct if we knew that every student's GPA was unique

Any operator

College(cName, state, enr)

Student(sID, sName, GPA, sizeHS)

Apply(sID, cName, major, decision)

Checks whether the value has a certain relationship with **at least one** element of the results of the subquery

```
SELECT sName
FROM Student
WHERE not GPA < any (select GPA from Student);
```

```
SELECT sName
FROM Student
WHERE GPA >= all (select GPA from Student);
```



$$\forall_{x \in X} \neg P(x) \equiv \neg \exists_{x \in X} P(x)$$

sName
Amy
Doris
Irene
Amy

Any operator – example 2

College(cName, state, enr)

Student(sID, sName, GPA, sizeHS)

Apply(sID, cName, major, decision)

Find all students who are not from the smallest high school in the database

```
SELECT sID, sName, sizeHS
FROM Student
WHERE sizeHS > any (select sizeHS from Student);
```

SQLite does not support the any and all operators

Must rewrite the queries using exists and not exists

How can we rewrite this query using exists?

sID	sName	HS
123	Amy	1000
234	Doris	1500
345	Irene	500
456	Amy	1000
567	Edward	2000
789	Gary	800
987	Helen	800
876	Irene	400
765	Jay	1500
654	Amy	1000
543	Craig	2000

Any operator – example 2

College(cName, state, enr)

Student(sID, sName, GPA, sizeHS)

Apply(sID, cName, major, decision)

```
SELECT sID, sName, sizeHS
FROM Student S1
WHERE exists (select * from Student S2
              where S2.sizeHS < S1.sizeHS);
```

sID	sName	HS
123	Amy	1000
234	Doris	1500
345	Irene	500
456	Amy	1000
567	Edward	2000
789	Gary	800
987	Helen	800
876	Irene	400
765	Jay	1500
654	Amy	1000
543	Craig	2000

An *any* or *all* query can always be written using *exists* or *not exists*

Any operator – example 3

College(cName, state, enr)

Student(sID, sName, GPA, sizeHS)

Apply(sID, cName, major, decision)

Can we rewrite the query that finds students who have applied to a major in CS and have not applied to a major in EE?

```
SELECT sID, sName
FROM Student
WHERE sID in (select sID from Apply where major='CS') AND
        sID not in (select sID from Apply where major='EE');
```

sID	sName
987	Helen
876	Irene
543	Craig

```
SELECT sID, sName
FROM Student
WHERE sID = any (select sID from Apply where major = 'CS') AND
        sID <> any (select sID from Apply where major = 'EE');
```

sID	sName
123	Amy
345	Craig
987	Helen
876	Irene
543	Craig

Satisfied if there's anybody who applied to EE that is not the same as the student we're looking at

Any operator – example 3

College(cName, state, enr)

Student(sID, sName, GPA, sizeHS)

Apply(sID, cName, major, decision)

```
SELECT sID, sName
FROM Student
WHERE sID = any (select sID from Apply where major = 'CS') AND
      sID <> any (select sID from Apply where major = 'EE');
```

sID	sName
123	Amy
345	Craig
987	Helen
876	Irene
543	Craig

How can we correct it?

```
SELECT sID, sName
FROM Student
WHERE sID = any (select sID from Apply where major = 'CS') AND
      not sID = any (select sID from Apply where major = 'EE');
```

sID	sName
987	Helen
876	Irene
543	Craig

Operators' summary

attribute [**NOT**] **IN** Relation

[**NOT**] attribute **IN** Relation

[**NOT**] **EXISTS** Relation

attribute <comparison> **ALL** Relation

ALL and ANY not supported by SQLite

attribute <comparison> **ANY** Relation

Comparison may be: <; >; <=; >=; =; <>

Existential quantifier

College(cName, state, enr)

Student(sID, sName, GPA, sizeHS)

Apply(sID, cName, major, decision)

An existential quantifier ($\exists$) is a quantifier of the form “there exists”

Find colleges with some applications of students with a GPA higher than 3.8

```
SELECT DISTINCT cName
FROM Apply, Student
WHERE Apply.sID=Student.sID AND GPA>3.8;
```

Existential is easy!

cName
Stanford
Berkeley
MIT
Cornell

Universal quantifier

College(cName, state, enr)

Student(sID, sName, GPA, sizeHS)

Apply(sID, cName, major, decision)

A universal quantifier ($\forall$) is a quantifier of the form “for all”

Find colleges with applications of students all having a GPA higher than 3.8



Equivalent

Find colleges that only have applications of students with a GPA higher than 3.8

```
SELECT DISTINCT cName
```

```
FROM Apply
```

```
WHERE Apply.cName NOT IN (
```

```
    SELECT Apply.cName FROM Apply, Student
```

```
    WHERE Apply.sID=Student.sID AND GPA<=3.8);
```

cName

Wooclap time!

Any doubts?

Readings

Jeffrey Ullman, Jennifer Widom, A first course in Database Systems 3rd Edition

- Section 6.1 – Simple Queries in SQL

- Section 6.2 – Queries Involving More Than One Relation

- Section 6.3 - Subqueries

- Section 6.4 – Full-Relation Operations

- Section 6.5 – Database Modifications

Philip Greenspun, SQL for Web Nerds, <http://philip.greenspun.com/sql/>